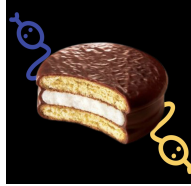
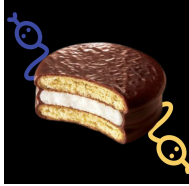


DB & SQL 코딩테스트 실전대비

세션 목차

1. DBMS의 종류 및 장단점
2. 카드 데이터의 기본 구성 요소
3. 카드 데이터로 알아보는 SQL 문법
4. SQL 코딩테스트 대비 문제 풀이

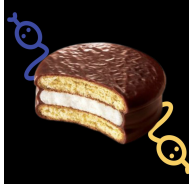
데이터와 정보



- 데이터 : 관찰의 결과로, 정량적 혹은 정성적인 실제 값을 의미
- 정보 : 데이터를 가공하여 의미를 부여한 것

e.g.	데이터	정보
	전세계 오라클 사용 점수 : 1236.37	오라클은 세계에서 가장 많이 사용되는 DBMS다

데이터 베이스란?



1. 데이터의 집합

데이터베이스란 여러 사람이 공유하고 사용할 목적으로 통합 관리되는 정보(Data)의 집합니다. 연관된 데이터의 모음을 구조화함으로써 검색 및 갱신의 편의를 추구한다.

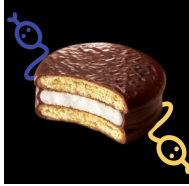
2. 중복이 없다.

데이터베이스 내에서는 기본적으로 **자료 항목**의 중복이 제거된 데이터를 추구한다. 사용자는 목적에 따라 동일한 데이터여도 다르게 사용할 수 있다. 이렇게 업무를 진행할 경우, 각각의 업무 전용 데이터 파일을 사용하더라도 각 파일에 중복된 정보가 들어있게 된다. 이 중복을 피해서 저로 관련성이 있는 데이터를 일원화하여 처리 효율성을 증대하고, 중복이 없는 데이터의 집합이 데이터베이스다.

즉, 데이터베이스(Database, DB)는 여러 데이터 파일을 조직적으로 통합하여 자료 항목의 중복을 없애고 자료를 구조화하여 기록해 놓은 데이터의 집합체다.

※ 의도적으로 중복을 허용하는 경우, 항상 파악하여 관리

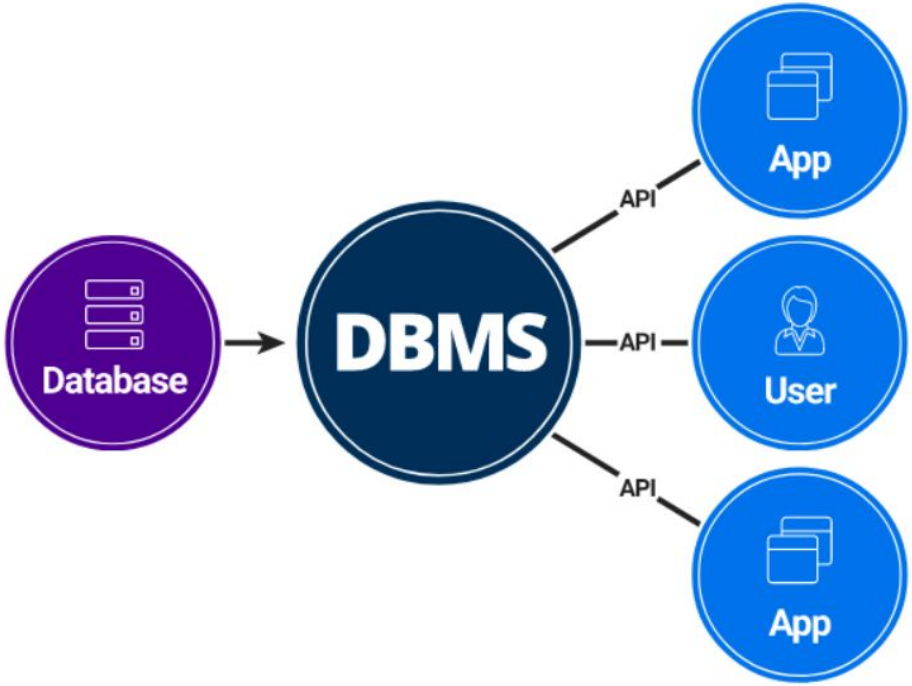
데이터베이스의 대표적인 특징



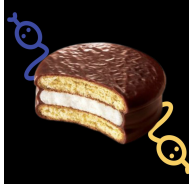
- **실시간 접근성(Real-time Accessibility)** : 사용자의 질의(Query)에 대해 즉시 처리하여 응답
- **계속적인 진화(Continuous evolution)** : CRUD(Create-생성, Read-읽기, Update-갱신, Delete-삭제)를 통해 항상 정확한 최신 데이터를 동적으로 유지
- **동시 공유(Concurrent sharing)** : 여러 사용자가 동시에 원하는 데이터를 공유
- **내용에 의한 참조(Content reference)** : 데이터베이스에 있는 데이터 참조 시, 튜플(Tuple)의 주소나 위치가 아닌 사용자가 원하는 데이터 내용에 따라 참조
- **데이터 논리적 독립성(Independence)** : 응용프로그램과 데이터베이스를 독립시킴으로써 데이터 논리적 구조를 변경하더라도 응용프로그램에는 반영되지 않음

DBMS란

데이터베이스 관리 시스템(DataBase Management System, DBMS)은 데이터베이스에서 데이터를 추출, 조작,정의,제어 등을 할 수 있게 해주는 데이터베이스 전용 관리 프로그램이다.



DBMS 장점



- **데이터 독립화** : 데이터와 응용 프로그램을 분리시킴으로써 상호 영향 정도 감소
- **데이터 중복 최소화/데이터 무결성 보장** : 중복되는 데이터를 최소화 시키면 데이터 무결성이 손상될 가능성 감소 및 저장공간의 여유 확보
- **데이터 보안 향상** : 응용프로그램은 DBMS를 통해 DBMS가 허용하는 데이터에만 접근 가능하고, 권한에 맞게 데이터 접근을 제한하거나 데이터를 암호화시켜 저장 가능
- **관리 편의성 향상** : 다양한 방법으로 데이터를 백업 및 장애 발생 시 데이터를 복구 가능

DMBS의 종류와 장단점

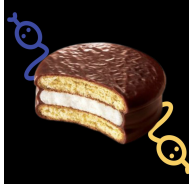
DBMS 종류	구조 / 개념 요약	장점	단점	실제 사용 DB 예시	실제 사용 사례
계층형 DBMS (HDBMS)	트리(Tree) 구조의 계층적 데이터 모델 부모–자식 간 **1:N** 관계	• 데이터 접근 속도 빠름 • 구조가 단순해 사용량 예측 용이	• 구조 변경이 매우 어려움 • 유연한 관계 표현 불가	IBM IMS	• 초기 은행 전산 시스템 • 통신사 가입자 관리(과거)
망형 DBMS (NDBMS)	네트워크(Graph) 구조 노드 간 **N:M 관계** 표현 가능	• 계층형의 상하 종속 문제 개선 • 복잡한 관계 표현 가능	• 설계·유지보수 난이도 높음 • 데이터 종속성 문제 여전	IDMS	• 항공 예약 시스템(과거) • 제조 BOM 구조(레거시)
관계형 DBMS (RDBMS)	2차원 테이블 기반 PK–FK로 테이블 간 관계 정의	• 업무 변화에 매우 강함 • SQL 표준화, 유지보수 용이 • 트랜잭션 안정성	• 대용량 처리 시 성능 한계 • 스케일 아웃 비용 큼	**Oracle**, **MySQL**	• 금융·카드사 핵심 DB • ERP / 인사·회계 시스템
객체지향 DBMS (ODBMS)	객체(Object)를 DB에 직접 저장 클래스·상속·메서드 개념 지원	• 복잡한 객체 구조 표현 가능 • 멀티미디어 데이터 처리 유리	• 표준 부족 • RDBMS 대비 생태계 약함	Oracle Object Type	• CAD/CAM • 멀티미디어 콘텐츠 관리
컬럼 기반 분석형 DB (OLAP)	컬럼 단위 저장 대규모 집계·분석 최적화	• 대용량 분석 매우 빠름 • 스토리지 효율 우수	• 트랜잭션 처리 부적합 • 실시간 쓰기 약함	**BigQuery**	• 로그/마케팅 분석 • BI 대시보드
NoSQL (비관계형)	스키마 유연 Key-Value / Document / Graph 등	• 수평 확장 용이 • 대규모 트래픽 처리	• JOIN 불가 또는 제한 • 트랜잭션 제약	MongoDB, Redis	• 세션 관리 • 실시간 추천/캐시

DMBS의 종류와 장단점

DBMS 종류	구조 / 개념 요약	장점	단점	실제 사용 DB 예시	실제 사용 사례
계층형 DBMS (HDBMS)	트리(Tree) 구조의 계층적 데이터 모델 부모–자식 간 **1:N** 관계	• 데이터 접근 속도 빠름 • 구조가 단순해 사용량 예측 용이	• 구조 변경이 매우 어려움 • 유연한 관계 표현 불가	IBM IMS	• 초기 은행 전산 시스템 • 통신사 가입자 관리(과거)
망형 DBMS (NDBMS)	네트워크(Graph) 구조 노드 간 **N:M 관계** 표현 가능	• 계층형의 상하 종속 문제 개선 • 복잡한 관계 표현 가능	• 설계·유지보수 난이도 높음 • 데이터 종속성 문제 여전	IDMS	• 항공 예약 시스템(과거) • 제조 BOM 구조(레거시)
관계형 DBMS (RDBMS)	2차원 테이블 기반 PK–FK로 테이블 간 관계 정의	• 업무 변화에 매우 강함 • SQL 표준화, 유지보수 용이 • 트랜잭션 안정성	• 대용량 처리 시 성능 한계 • 스케일 아웃 비용 큼	Oracle, MySQL	• 금융·카드사 핵심 DB • ERP / 인사·회계 시스템
객체지향 DBMS (ODBMS)	객체(Object)를 DB에 직접 저장 클래스·상속·메서드 개념 지원	• 복잡한 객체 구조 표현 가능 • 멀티미디어 데이터 처리 유리	• 표준 부족 • RDBMS 대비 생태계 약함	Oracle Object Type	• CAD/CAM • 멀티미디어 콘텐츠 관리
컬럼 기반 분석형 DB (OLAP)	컬럼 단위 저장 대규모 집계·분석 최적화	• 대용량 분석 매우 빠름 • 스토리지 효율 우수	• 트랜잭션 처리 부적합 • 실시간 쓰기 약함	BigQuery	• 로그/마케팅 분석 • BI 대시보드
NoSQL (비관계형)	스키마 유연 Key-Value / Document / Graph 등	• 수평 확장 용이 • 대규모 트래픽 처리	• JOIN 불가 또는 제한 • 트랜잭션 제약	MongoDB, Redis	• 세션 관리 • 실시간 추천/캐시

DBMS의 종류와 장단점

운영계, 분석계, 개발계



[운영계 OLTP]
- 주문 / 결제 / 계좌
- Oracle / MySQL

↓ (ETL, CDC)

[분석계 OLAP]
- 매출 / 고객 분석
- BigQuery

[개발계]
- 운영계 구조 복사
- 테스트 / 검증용

구분	의미
운영계	실제 비즈니스가 돌아가는 시스템
분석계	운영 데이터를 활용해 인사이트를 뽑는 시스템
개발계	테스트·검증·실험 환경 (성격이 조금 다름)

OLTP vs OLAP

구분	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
목적	실시간 **업무 처리**	대규모 **분석·의사결정 지원**
주요 사용자	일반 사용자, 운영 담당자	데이터 분석가, 기획자, 경영진
데이터 성격	현재 중심, 실시간 데이터	과거 누적 데이터
데이터 양	상대적으로 적음 (건 단위)	매우 큼 (수백 GB ~ PB)
쿼리 유형	INSERT / UPDATE / DELETE 중심 짧고 단순한 쿼리	SELECT + JOIN + 집계 중심 길고 복잡한 쿼리
처리 단위	단일 레코드 또는 소수 레코드	대량 레코드 스캔
정합성(ACID)	매우 중요 (필수)	상대적으로 덜 중요
응답 속도 기준	밀리초(ms) 단위	초~분 단위 허용
스키마 구조	정규화(Normalization)	비정규화(Denormalization), 스타 스키마
확장 방식	Scale-up 중심	Scale-out 중심
대표 DBMS	Oracle, MySQL	BigQuery, Redshift, Snowflake
대표 사용 예시	주문 처리, 결제, 계좌 이체	매출 분석, 고객 행동 분석, KPI 대시보드

OLTP
→ *"지금 이 순간의 업무를 정확하게 처리하는 시스템"*
(운영계)

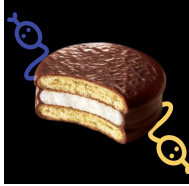
→ *짧은 쿼리, 소량 데이터, 잦은 쓰기*

OLAP
→ *"과거 데이터를 모아 의미를 뽑아내는 시스템"*
(분석계)

→ *긴 쿼리, 대량 스캔, 집계 중심*

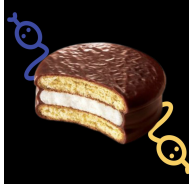
DBMS의 종류와 장단점

DB별 SQL 차이

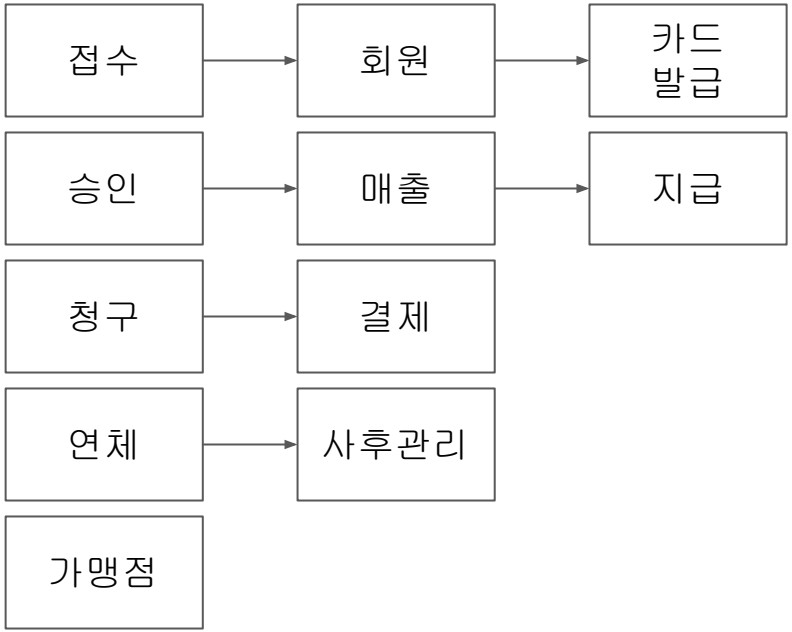


구분	Oracle	MySQL	PostgreSQL	BigQuery	Redshift	Snowflake
계열	OLTP (RDBMS)	OLTP (RDBMS)	OLTP+OLAP	OLAP (DW)	OLAP (DW)	OLAP (DW)
SQL 성격	전통적, 엄격	실용적, 관대	표준 SQL에 가장 충실	분석 특화	Postgres 기반	ANSI SQL 중심
LIMIT 문법	❌ (ROWNUM)	✅ LIMIT	✅ LIMIT	✅ LIMIT	✅ LIMIT	✅ LIMIT
날짜 처리	복잡, 강력	단순	표준적	함수 중심	유사 Postgres	직관적
서브쿼리	강력	제한적	매우 강력	분석 최적화	분석용	분석용
윈도우 함수	강력	제한적	매우 강력	필수	필수	필수
MERGE / UPSERT	MERGE	INSERT ON DUPLICATE	ON CONFLICT	MERGE	MERGE	MERGE
트랜잭션	매우 강함	강함	매우 강함	❌	제한적	제한적
대량 스캔	❌	❌	△	✅	✅	✅

카드 거래 데이터의 기본 구성 요소

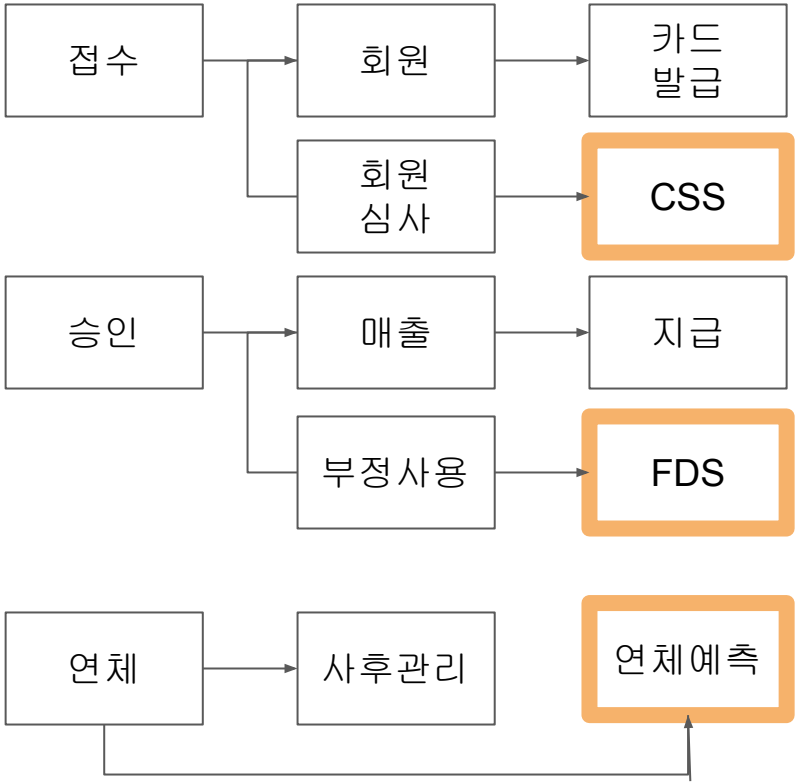


- **고객**
- **매출**
- **결제**
- **연체**
- **가맹점**

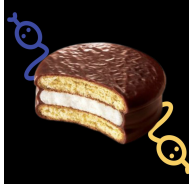


카드 거래 데이터의 기본 구성 요소

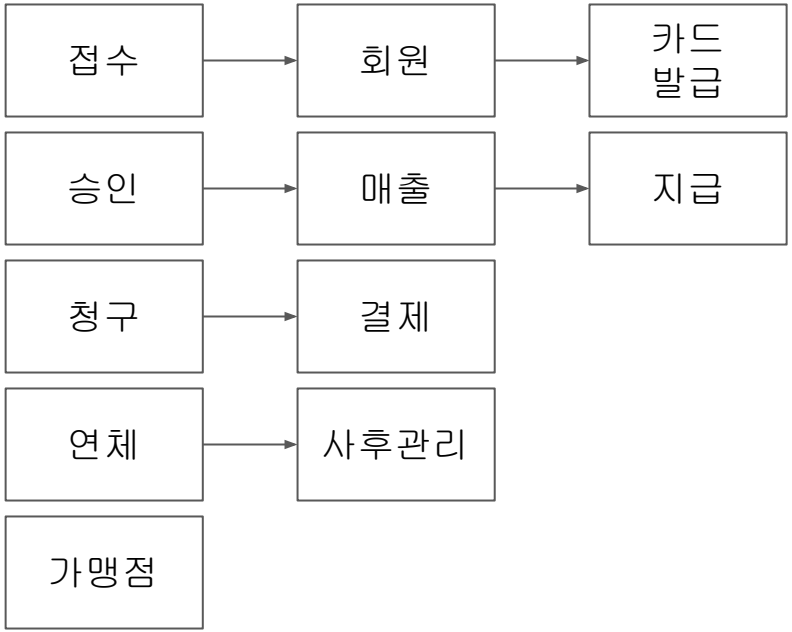
- **고객**
- **매출**
- **연체**



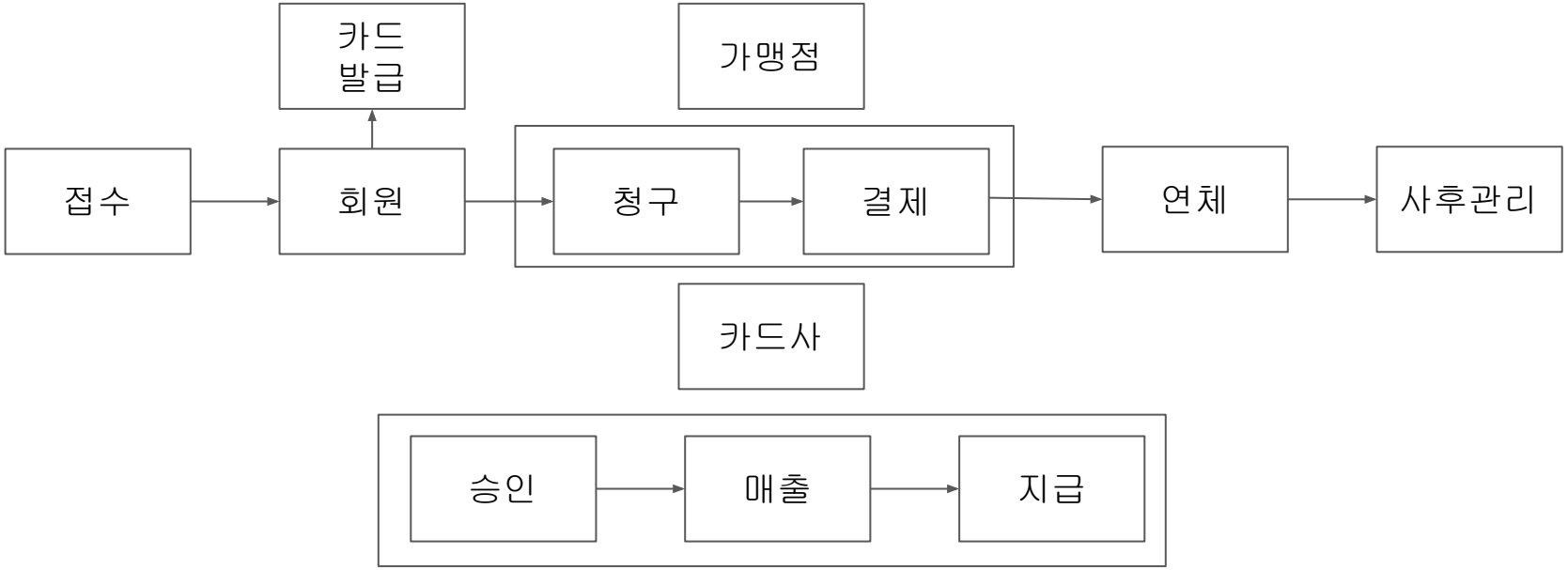
카드 데이터로 알아보는 SQL 문법



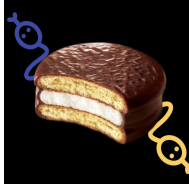
- **고객**
- **매출**
- **결제**
- **연체**
- **가맹점**



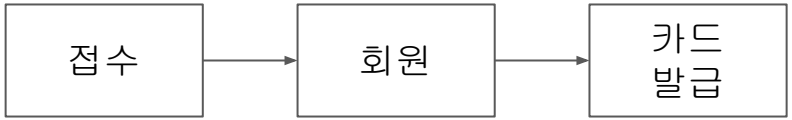
카드 데이터로 알아보는 SQL 문법



카드 데이터로 알아보는 SQL 문법

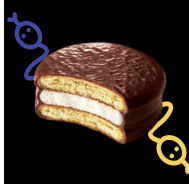


- **고객**



- 개인정보 이슈 : 마케팅, 분석 목적으로 사용하기 위한 개인 비식별화 필요
  - 고객번호
  - 회원번호
  - 카드번호
  - 주민번호대체ID

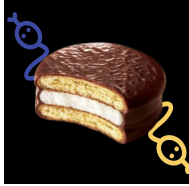
카드 데이터로 알아보는 SQL 문법



Data Mart란? 테이블의 적제요건에 따라 집계된 테이블

- 고객
  - 고객별 월실적
  - 고객별 일실적
  - 고객 세그먼트 분류 (신규 유치회원, 프리미엄 고객 등)
- 매출
  - 업종별 월실적
  - 업종별 일실적
- 가맹점
  - 가맹점별 월실적
  - 가맹점별 일실적

카드 데이터로 알아보는 SQL 문법



SQL로 하는 일? 새로운 집합 생성!

- 고객
- 회원
- 매출

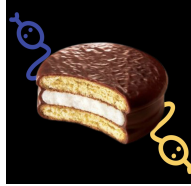
고객번호 = 고객번호 회원번호 → 회원에 존재하는 고객집합

조인

고객번호 회원번호 → 매출이 존재하는 고객집합

GROUP BY 매출년월 → 월별 매출액 집합

카드 데이터로 알아보는 SQL 문법

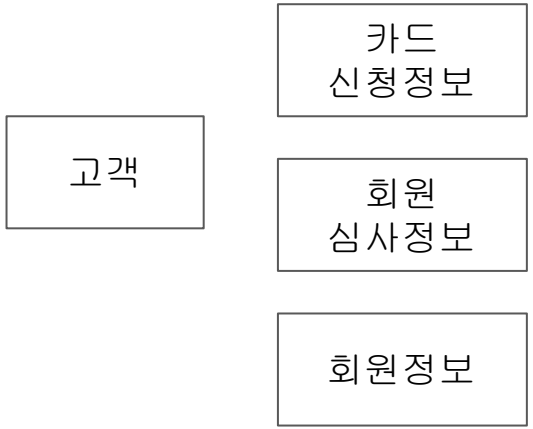


SQL 의 작성순서

SQL 절	사용용도
SELECT	컬럼, 함수, 계산식 등의 조회
FROM	조회하고 싶은 테이블을 입력
WHERE	검색조건, 계산식 등
GROUP BY	특정 컬럼등을 기준으로 그룹 연산
HAVING BY	집합을 기준으로 조건 부여
ORDER BY	특정 컬럼을 기준으로 순서 부여

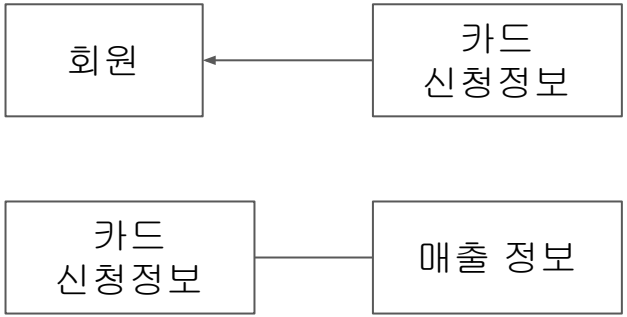
- **Dimension : 분석 관점 -> 연령, 결제일**
- **Fact : 분석 계수 -> 건수, 고객수, 금액 등**
- **Condition : 테이블 탐색 제한 조건**

카드 데이터로 알아보는 SQL 문법



- Key 컬럼(=primary key)은 테이블에서 유일성을 보장해줌
- 각 테이블에서 join 할 시에 Key 컬럼을 유의해서 사용해야함

카드 데이터로 알아보는 SQL 문법

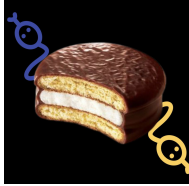


1 : Many -> 대부분 RDB 관계

Many : Many -> Cartesian Product

- 각 테이블의 관계를 이해해야 올바른 쿼리를 작성할 수 있음
- 각 테이블에서 join 할 시에 Many:Many 의 경우 적절하지 않은 결과를 초래함

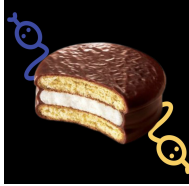
카드 데이터로 알아보는 SQL 문법



```
SELECT 성별
          , 연령
          , sum(매출금액) as  매출액
          , count(*)  as 매출건수
  FROM 회원정보        T0
          , 매출정보 T1
WHERE T0.회원번호 = T1.회원번호
     AND 기준년월 BETWEEN '202301' AND '202306'
GROUP BY 성별
                ,연령
```

- GROUP BY 1 원칙 : SELECT 절에 지정된 디멘전 컬럼은 GROUP BY 절에 모두 지정되어야 한다.
- GROUP BY 2 원칙 : 디멘전 컬럼이 아닌 컬럼의 경우 반드시 집계 함수가 있어야 한다.
- GROUP BY 3 원칙 : GROUP BY된 결과 행수는 디멘전 컬럼의 카다널리티(경우의 수)에 의해 결정된다
- GROUP BY 4 원칙 : GROUP BY 결과 행수는 이후 SELECT 구문에서 변경되지 않는다.
- GROUP BY 5 원칙 : 제 4원칙의 결과 행수는 모집할 행수보다 반드시 작다 (KEY칼럼으로 자르기 제외)

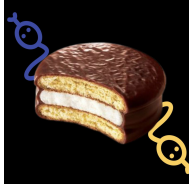
카드 데이터로 알아보는 SQL 문법



- GROUP BY 1 원칙 : SELECT 절에 지정된 디멘전 컬럼은 GROUP BY 절에 모두 지정되어야 한다.

```
SELECT 성별
          , 연령
          , sum(매출금액) as  매출액
          , count(*)  as 매출건수
  FROM 회원정보        T0
          , 매출정보 T1
WHERE T0.회원번호 = T1.회원번호
     AND 기준년월 BETWEEN '202301' AND '202306'
GROUP BY 성별
              ,연령
```

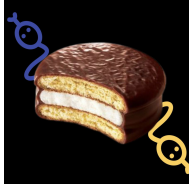
카드 데이터로 알아보는 SQL 문법



- **GROUP BY 2 원칙 : 디멘전 컬럼이 아닌 컬럼의 경우 반드시 집계 함수가 있어야 한다.**

```
SELECT 성별
          , 연령
          , sum(매출금액) as  매출액
          , count(*)  as 매출건수
  FROM 회원정보        T0
          , 매출정보 T1
WHERE T0.회원번호 = T1.회원번호
     AND 기준년월 BETWEEN '202301' AND '202306'
GROUP BY 성별
                ,연령
```

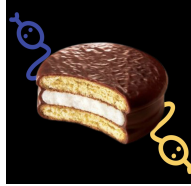
카드 데이터로 알아보는 SQL 문법



- GROUP BY 3 원칙 : GROUP BY된 결과 행수는 디멘전 컬럼의 카다널리티(경우의 수)에 의해 결정된다

```
SELECT 성별
          , 연령
          , sum(매출금액) as  매출액
          , count(*)  as 매출건수
  FROM 회원정보        T0
          , 매출정보 T1
WHERE T0.회원번호 = T1.회원번호
     AND 기준년월 BETWEEN '202301' AND '202306'
GROUP BY 성별
                ,연령
```

카드 데이터로 알아보는 SQL 문법



SQL 수행순서

FROM

WHERE

GROUP BY

HAVING BY

SELECT

ORDER BY

적을땐 요
순서대로

FROM

WHERE

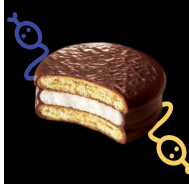
GROUP BY

HAVING BY

SELECT

ORDER BY

카드 데이터로 알아보는 SQL 문법



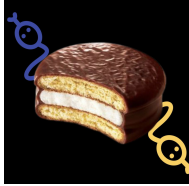
- **집합내 특정 멤버를 찾을 때 -> WHERE**

```
SELECT 회원번호
  FROM 회원정보        T0
WHERE 기준년월 = '202301'
```

- **다수의 맴버를 보유하고 있는 특정 집합을 찾을때 -> HAVING**

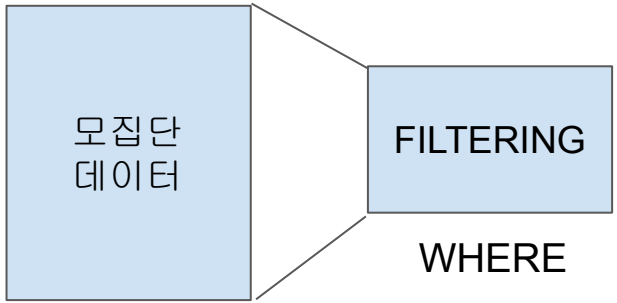
```
SELECT 회원번호
   FROM 매출정보 T0
WHERE 기준년월 = '202301'
GROUP BY 회원번호
HAVING MIN(매출액) >= 10만원 & SUM(매출액) >= 100만원
```

카드 데이터로 알아보는 SQL 문법

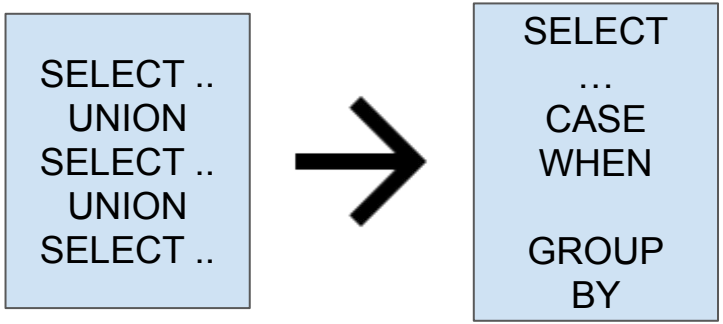


1. What을 먼저 생각한다 => 필요한 집합이 무엇인지 먼저 생각한다
2. 그러한 결과를 얻기 위해 어떤 집합이 필요한지 생각한다
3. 조인 경로를 최대한 줄인다 => 테이블 이해도가 높아야함
4. 탐색횟수를 최대한 줄인다

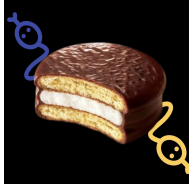
- **모집단 데이터 최대한 압축**



- **테이블 탐색 횟수 최대한 축소**



카드 데이터로 알아보는 SQL 문법

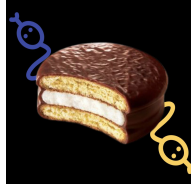


JOIN

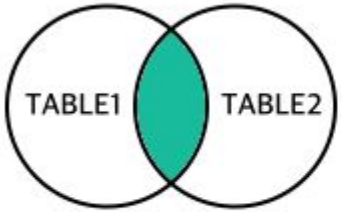
- 두개 이상의 테이블에서 원하는 데이터를 검색할 때 사용
- 최소한의 JOIN 조건은 ‘테이블 개수 -1개’ 이므로 그 이상의 JOIN 조건이 필요,

  그렇지 않으면 Catesian Product가 발생
- 테이블 ALIAS (= 별칭)를 사용 -> 데이터 엑세스를 빠르게 해줌
- INNER JOIN -> 비교하는 테이블에서 JOIN 컬럼의 값들이 정확하게 일치하는 경우
- OUTER JOIN -> 비교하는 테이블에서 JOIN 컬럼 값들이 서로 일치하지 않는 경우

카드 데이터로 알아보는 SQL 문법

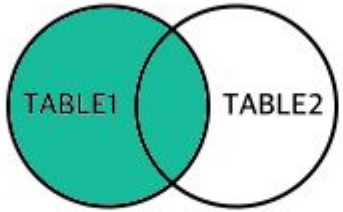


(INNER) JOIN



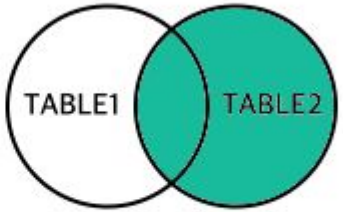

```
SELECT *
FROM table1 A
JOIN table2 B
ON A.key = B.key
```

LEFT (OUTER) JOIN



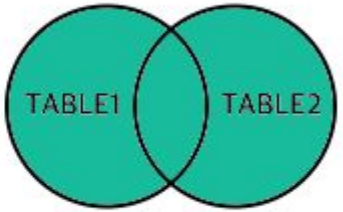

```
SELECT *
FROM table1 A
LEFT JOIN table2 B
ON A.key = B.key
```

RIGHT (OUTER) JOIN




```
SELECT *
FROM table1 A
RIGHT JOIN table2 B
ON A.key = B.key
```

FULL (OUTER) JOIN




```
SELECT *
FROM table1 A
FULL JOIN table2 B
ON A.key = B.key
```

Left Table

Date	CountryID	Units
1/1/2024	1	40
1/2/2024	1	25
1/3/2024	3	30
1/4/2024	2	35

Right Table

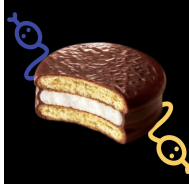
ID	Country
3	Panama
4	Spain

Merged Table

Date	CountryID	Units	Country
1/3/2024	3	30	Panama

출처 :
https://learn.microsoft.com/ko-kr/power-query/merge-queries-inner
https://www.devkuma.com/docs/sql/join/

카드 데이터로 알아보는 SQL 문법



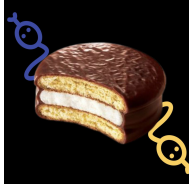
회원

회원번호	연령	입회일자
1	20	20150512
2	20	20151220
3	30	20161017

매출

회원번호	매출일자	매출금액
1	20111005	100
1	20120312	200
1	20130814	300
2	20170612	400
2	20170618	500
0	20171005	600

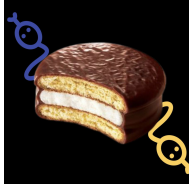
카드 데이터로 알아보는 SQL 문법



회원 X 매출 (inner join)

회원번호	연령	입회일자	매출일자	매출금액
1	30	20110512	20111005	100
1	30	20110512	20120312	200
1	30	20110512	20130814	300
2	40	20151220	20170612	400
2	40	20151220	20170618	500

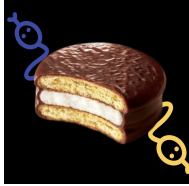
카드 데이터로 알아보는 SQL 문법



회원 X 매출 (outer join)

회원번호	연령	입회일자	매출일자	매출금액
1	30	20110512	20111005	100
1	30	20110512	20120312	200
1	30	20110512	20130814	300
2	40	20151220	20170612	400
2	40	20151220	20170618	500
3	30	20161017	NULL	NULL

카드 데이터로 알아보는 SQL 문법



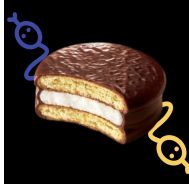
Cartesian Product

- ➔ 2개 이상의 테이블을 Join하여 조회할 경우 1번째 테이블의 모든 행들이 2번째 테이블의 모든 행과 join되는 형태 (M*N행의 결과가 리턴)
- ➔ 발생하는 경우는
  - ◆ JOIN 조건을 생략하는 경우
  - ◆ JOIN 조건을 중복하는 경우 (Many : Many의 관계에서)
- ➔ 예시

카드 신청정보 — 매출 정보

Many : Many -> Cartesian Product

카드 데이터로 알아보는 SQL 문법



WITH

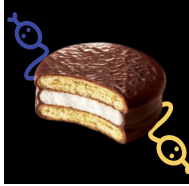
```
WITH TB1 as (
 SELECT 회원번호
          , SUM(매출액)
   FROM 매출정보 T0
WHERE 기준년월 = '202301'
GROUP BY 회원번호
)

SELECT A.*
   FROM TB1 A
```

SUB QUERY

```
SELECT A.*
   FROM (
 SELECT 회원번호
          , SUM(매출액)
   FROM 매출정보 T0
WHERE 기준년월 = '202301'
GROUP BY 회원번호
) A
```

카드 데이터로 알아보는 SQL 문법



효율적인 Join 방법

1. 붙이려는 테이블을 쪼개서 붙인다. (WITH, Sub Query 등 사용)
2. 붙이려는 테이블의 조건을 WHERE절이 아닌 JOIN 조건절에 사용한다

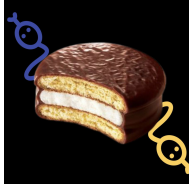
- **연체건수를 찾고 싶을때**

```
SELECT COUNT(*)
  FROM 카드기본 T0
   LEFT JOIN 카드연체 T1
      ON T0.고객번호 = T1.
고객번호
WHERE T0.기준년월 = '202301'
    AND T1.연체여부 = 1
```

→

```
SELECT COUNT(*)
  FROM 카드기본 T0
   INNER JOIN 카드연체 T1
      ON T0.고객번호 = T1.
고객번호
      ON T1.연체여부 = 1
WHERE T0.기준년월 = '202301'
```

카드 데이터로 알아보는 SQL 문법



CASE

업종코드 테이블에서 업종명에 ‘호텔’이 들어가면 호텔, ‘항공’이 들어가면 항공 그외는 기타로 구분하여 건수를 산출

```
SELECT CASE WHEN 업종명 LIKE ‘%호텔%’ THEN ‘호텔’
            WHEN 업종명 LIKE ‘%항공%’ THEN ‘항공’
            ELSE ‘기타’
       END AS 구분
     , COUNT(*) AS 건수
  FROM 업종코드테이블
GROUP BY  CASE WHEN 업종명 LIKE ‘%호텔%’ THEN ‘호텔’
               WHEN 업종명 LIKE ‘%항공%’ THEN ‘항공’
               ELSE ‘기타’
          END
```

감사합니다.

Copyright. 2024. Teamsparta. All rights reserved.
무단 배포를 금지합니다.